

TRENCH-IDS

Transferable REpresentationN learning for Continual Heterogeneous graph-based IDS

Project Summary

2026-07-24

9 / 10

steps implemented

0.86

replay final accuracy

0.09

replay avg. forgetting

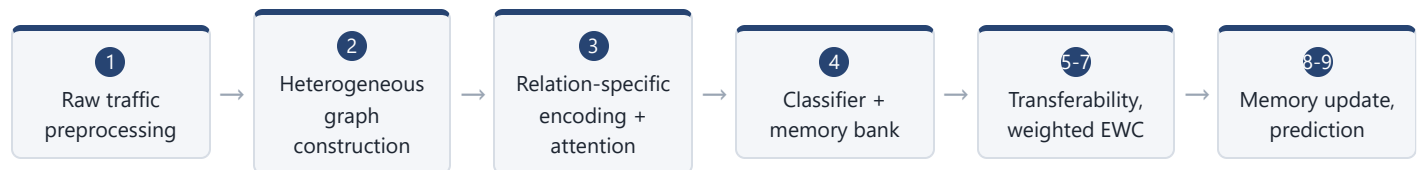
3

datasets, 10 classes, 6 tasks

Problem and goal

A continual IDS classifier trained sequentially on new attack classes tends to forget earlier ones, and current CL methods have no way to tell which learned representations are actually worth protecting for attacks it hasn't seen yet. TRENCH-IDS builds a heterogeneous-graph encoder with a separate embedding per relation type, a memory bank keyed on those relation embeddings, and a CL mechanism that weights EWC regularization by measured transferability. It's evaluated on a 6-task, 10-class benchmark built from three NetFlow v2 datasets (ToN-IoT, CSE-CIC-IDS2018, BoT-IoT).

Methodology pipeline



Steps 1 through 9 are implemented and have been run on real data. Step 10 (a standalone transferability and unseen-traffic prediction deliverable) is partly done; see page 4.

Task sequence (T1 through T6, similarity-driven isolate-and-bundle)



Benign traffic is present in every task, resampled per task at a fixed 3:1 attack-to-benign ratio (see "Setup" on page 4). Classes are grouped by *low* pairwise similarity on purpose, so the model can't shortcut transfer by learning two similar attacks from labels in the same task.

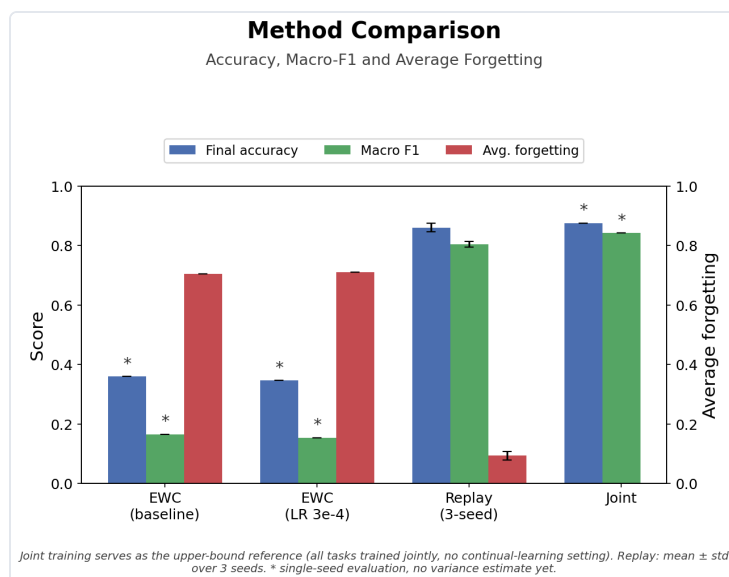
Headline result

Experience replay brings average forgetting down from about 0.71 (plain fine-tuning, and every Online EWC variant tested) to 0.094 ± 0.015 (mean $\pm$ std, 3 seeds), and final accuracy up from 0.36 to 0.861 ± 0.014 , about 1.5 points below the joint-training upper bound of 0.876. Full numbers and figures on the next pages.

Method comparison

Method	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)	F1 (weighted)	Forgetting
Plain fine-tuning, no EWC	0.356	–	–	–	–	0.711
Online EWC, uniform ($\lambda=1$)	0.359	–	–	–	–	0.710
Online EWC, relation-aware ($\lambda=1$)	0.362	0.217	0.268	0.166	0.208	0.705
Online EWC, λ sweep 0.01 to 1000 (3 of 6 configs with full eval)	0.24 to 0.34	0.074 to 0.205	0.204 to 0.260	0.10 to 0.17	0.138 to 0.198	0.710 to 0.798
Experience replay (3 seeds, mean $\pm$ std)	0.861 $\pm$ 0.014	0.793 $\pm$ 0.026	0.854 $\pm$ 0.027	0.805 $\pm$ 0.010	0.847 $\pm$ 0.018	0.094 $\pm$ 0.015
Joint training (upper bound, no CL setting)	0.876	0.825	0.880	0.842	0.875	–

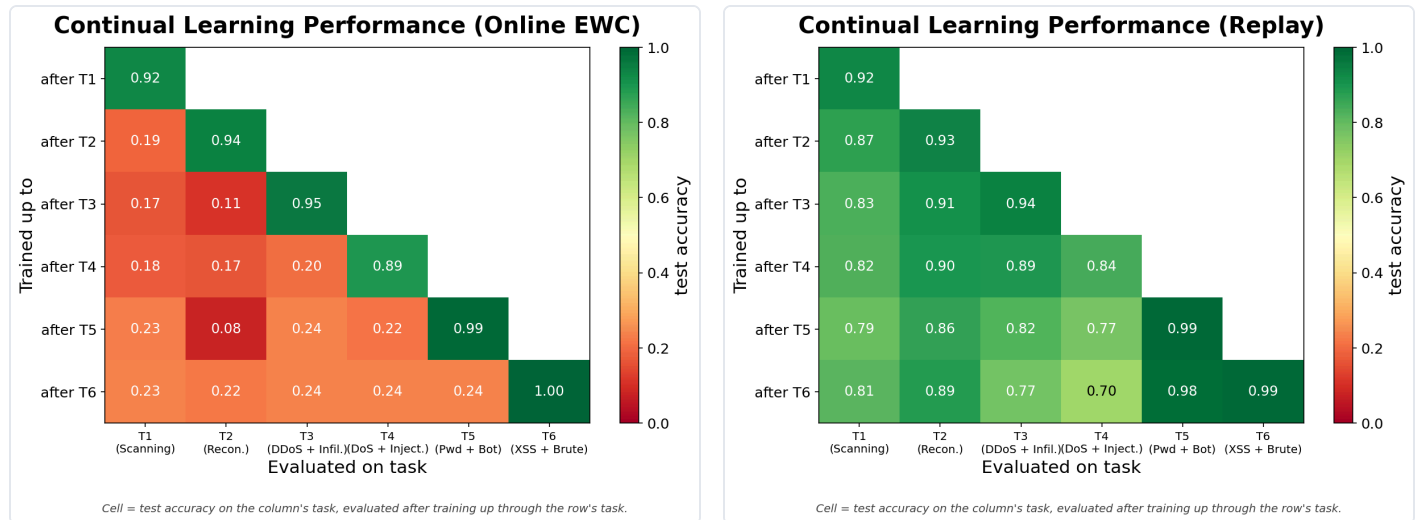
Accuracy here is the final average across tasks (matches the forgetting-matrix numbers used elsewhere in this report); precision, recall, and F1 are pooled over all test rows. The plain fine-tuning and uniform-EWC rows only have accuracy and forgetting logged; nobody has run the full precision/recall/F1 evaluation on those two checkpoints yet, which is why they're blank rather than zero. Every EWC row above is single-seed. Joint training is a non-continual reference: all 6 tasks trained together rather than sequentially, so it has no forgetting to report. Replay training took roughly 48 to 95 minutes per seed on the available GPU; joint training took about 50 minutes.



Why EWC doesn't help here

The negative result held up under scrutiny. A logarithmic λ sweep across five orders of magnitude (0.01 to 1000) produced no monotonic trend in forgetting. An instrumented run at $\lambda=1000$ showed the EWC penalty term actually exceeding the classification loss early in training, which rules out " λ was too small to matter." And a matched comparison at $\lambda=1$ (plain fine-tuning 0.711, uniform EWC 0.710, relation-aware weighted EWC 0.705) put all three within noise of each other. The learned relation-importance weights also collapse toward zero from task 2 onward: minimizing the training loss gives the model a direct incentive to shrink its own penalty term, and nothing in the per-task objective rewards keeping it large for later tasks.

Each cell is test accuracy on the column's task, measured after training has continued through the row's task. A method with little forgetting stays green all the way down a column; one that forgets fades to orange or red below the diagonal.



Under EWC, every earlier task drops to somewhere between 0.08 and 0.24 accuracy within a task or two. Under replay, every task stays between 0.70 and 0.99 through the end of training.

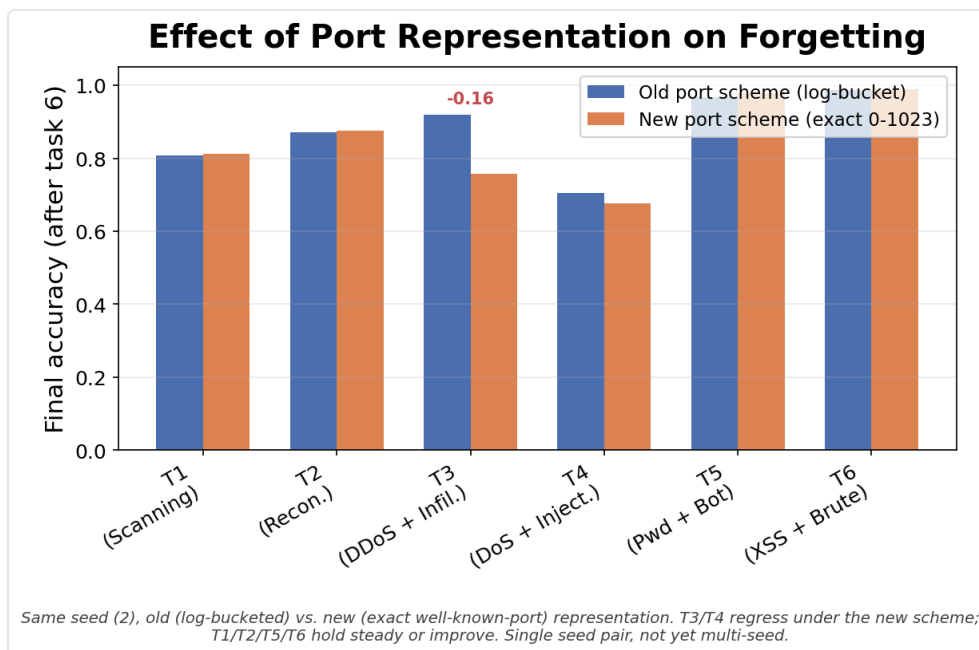
Per-class picture (replay, seed 42)

Class	Precision	Recall	F1	Support
Password	0.965	0.997	0.981	172,999
XSS	0.967	0.997	0.982	368,253
Bot	0.912	1.000	0.954	21,464
DDoS	0.843	0.986	0.909	512,468
Benign	0.910	0.528	0.668	784,427
BruteForce	0.546	0.983	0.702	18,137
Infiltration	0.170	0.422	0.242	17,370

Infiltration is the weak point: it's the smallest class by far (17,370 rows, 0.1% of pooled traffic) and gets confused mostly with DDoS (7,258 of its rows) and BruteForce (1,783). Benign's recall is also lower than its precision, meaning the model over-predicts attack classes on ambiguous traffic more than it misses real attacks, which is the safer direction for an IDS to err in.

Port representation fix

Fixed a bucket-collision bug where well-known ports (FTP, SSH, Telnet: ports 20 through 23) were sharing a single log-bucketed embedding row, a real information loss since port identity only reaches the model through this embedding. Ports 0 through 1023 now get exact per-port rows; the sparse 1024 to 65535 tail still uses log-bucketing. One controlled same-seed comparison (old scheme vs. new scheme) gives a mixed result:



T3 and T4 forgetting gets worse under the new scheme (T3 final accuracy down 0.16, T4 down 0.03), while T1, T2, T5, and T6 hold steady or improve slightly. This is one seed pair, not yet a robust result. It's worth a fuller multi-seed comparison before deciding whether to keep the fix as is, revert it, or scope it down, for example limiting exact embeddings to the top-K highest-traffic ports instead of the full 0 to 1023 range.

Setup

All numbers on this page and the previous three come from graphs built with **benign_ratio = 3.0** (data/graphs, the default in configs/graph.yaml), the same graph set used since Step 4. The benign-ratio sweep (2.0, 3.0, 4.0) was run earlier during Step 2 to check graph construction was reasonable across settings, but 3.0 is the only ratio any Step 4+ training run (EWC, replay, joint, or the port-scheme comparison) has actually used. Revisiting the other two ratios with the current model and training setup hasn't been done.

Status

- ✓ Steps 1 through 9 implemented and run end to end on real data, all 6 tasks.
- ✓ Replay works as a forgetting mitigation. EWC's ineffectiveness is a settled result at this point, not a sign of under-tuning.
- ✓ Port-representation bug fixed as requested. 121+ tests passing throughout.
- Step 10's dedicated transferability and unseen-traffic prediction deliverable: the pieces exist scattered across other steps' outputs, not yet pulled into one artifact.
- Multi-seed statistics for the EWC and hyperparameter-sweep runs (currently single-seed each).
- Port-scheme decision: keep, revert, or refine, pending more evidence than one seed pair.